super-careful, because that could lead to bugs and deadlocks[29].

7.7.2 Introducing Locks

When we get at line 4, though, we see a real new-entry, a brand-new attribute for our class: an object of type `ReentrantReadWriteLock`. Before explaining explaining what it is a reentrant lock, we should make one thing clear: this is the mechanism that we are going to use to handle concurrent access to `LRUCache`'s methods. Java provides several different ways to do so, from the declaring methods as synchronized to using semaphores, latches etc… of these mechanisms is the best depends, as always, on the context, and sometimes more than one method could work. In this case, I prefer using a reentrant *read/write* lock rather than declaring methods as synchronized because it gives us more flexibility in deciding when we should use a lock on read and when on write. As we'll see, there might be a big impact on performance if we don't get this right.

But first things first, we still need to define what a lock is, not to mention what reentrant means.

A lock is a concurrency mechanism whose name is pretty self-explanatory. You probably have already heard of database locks, and the principle is exactly the same for code. If an object (an instance of class) has some code wrapped inside a lock, this means that, for that instance, no matter how many calls are made to that method, the locked portion of code can only be executed by one call at the time – all the other will have to wait for their turn.